

Лабораторная работа №4. Продвинутое использование git

Дисциплина: Архитектура компьютеров и операционные системы

Игнатенко Денис Бенъяминович

Содержание

1	Цель работы	6
2	Задание	7
3	Теоретическое введение	8
3.1	Рабочий процесс Gitflow	8
3.2	Семантическое версионирование	8
3.3	Общепринятые коммиты (Conventional Commits)	9
4	Выполнение лабораторной работы	11
4.1	Установка программного обеспечения	11
4.1.1	Установка git-flow	11
4.1.2	Установка Node.js	12
4.1.3	Настройка Node.js	12
4.1.4	Установка commitizen и standard-changelog	13
4.2	Создание репозитория на GitHub	13
4.3	Локальная инициализация репозитория	14
4.4	Настройка conventional commits	15
4.4.1	Инициализация npm	15
4.4.2	Редактирование package.json	16
4.4.3	Первый коммит через git cz	17
4.4.4	Отправка на GitHub	17
4.5	Инициализация git-flow	18
4.5.1	Инициализация структуры git-flow	18
4.5.2	Проверка текущей ветки	18
4.5.3	Отправка всех веток на GitHub	18
4.6	Создание релиза 1.0.0	19
4.6.1	Создание ветки релиза	19
4.6.2	Создание журнала изменений	19
4.6.3	Завершение релиза	20
4.6.4	Отправка релиза на GitHub	20
4.6.5	Страница релизов на GitHub	21
4.7	Работа с feature-веткой	22
4.7.1	Создание feature-ветки	22
4.7.2	Добавление нового файла	22
4.7.3	Завершение feature-ветки	23

4.8	Создание релиза 1.2.3	23
4.8.1	Создание ветки релиза	23
4.8.2	Обновление версии в package.json	24
4.8.3	Обновление журнала изменений	24
4.8.4	Завершение релиза 1.2.3	24
4.8.5	Отправка релиза на GitHub	25
4.8.6	Страница релизов с обоими релизами	26
5	Выводы	27
	Список литературы	28

Список иллюстраций

4.1	Установка git-flow	11
4.2	Установка Node.js и npm	12
4.3	Настройка npm	12
4.4	Установка commitizen и standard-changelog	13
4.5	Страница созданного репозитория на GitHub	14
4.6	Локальная инициализация и отправка на GitHub	15
4.7	Инициализация npm	15
4.8	Содержимое package.json	16
4.9	Интерфейс git cz	17
4.10	Отправка изменений на GitHub	17
4.11	Инициализация git-flow	18
4.12	Проверка текущей ветки	18
4.13	Отправка всех веток на GitHub	19
4.14	Создание ветки релиза 1.0.0	19
4.15	Содержимое CHANGELOG.md	20
4.16	Завершение релиза 1.0.0	20
4.17	Отправка релиза на GitHub	21
4.18	Страница релизов на GitHub	21
4.19	Создание feature-ветки	22
4.20	Коммит через git cz	22
4.21	Завершение feature-ветки	23
4.22	Создание ветки релиза 1.2.3	23
4.23	Обновлённый package.json	24
4.24	Обновлённый CHANGELOG.md	24
4.25	Завершение релиза 1.2.3	25
4.26	Отправка релиза 1.2.3 на GitHub	25
4.27	Страница релизов с обоими релизами	26

Список таблиц

3.1	Типы коммитов Conventional Commits {#tbl:commit-types}	9
-----	--	---

1 Цель работы

Получение навыков правильной работы с репозиториями git.

2 Задание

1. Выполнить работу для тестового репозитория.
2. Преобразовать рабочий репозиторий в репозиторий с git-flow и conventional commits.

3 Теоретическое введение

3.1 Рабочий процесс Gitflow

Gitflow Workflow — это методология ветвления, опубликованная и популяризованная Винсентом Дриссенем. Она предполагает выстраивание строгой модели ветвления с учётом выпуска проекта и отлично подходит для организации рабочего процесса на основе релизов.

Последовательность действий при работе по модели Gitflow:

- Из ветки `master` создаётся ветка `develop`
- Из ветки `develop` создаётся ветка `release`
- Из ветки `develop` создаются ветки `feature`
- Когда работа над веткой `feature` завершена, она сливается с веткой `develop`
- Когда работа над веткой `release` завершена, она сливается в ветки `develop` и `master`
- Если в `master` обнаружена проблема, из `master` создаётся ветка `hotfix`
- Когда работа над веткой `hotfix` завершена, она сливается в ветки `develop` и `master`

3.2 Семантическое версионирование

Семантическое версионирование — это формальное соглашение для определения номера версии программного обеспечения. Версия задаётся в виде МАЖОРНАЯ.МИНОРНАЯ.ПАТЧ:

- **МАЖОРНУЮ** версию увеличивают, когда сделаны обратно несовместимые изменения API
- **МИНОРНУЮ** версию увеличивают, когда добавляется новая функциональность с сохранением обратной совместимости
- **ПАТЧ**-версию увеличивают, когда делаются обратно совместимые исправления

3.3 Общепринятые коммиты (Conventional Commits)

Спецификация Conventional Commits — это соглашение о том, как нужно писать сообщения коммитов. Она совместима с семантическим версионированием и регламентирует структуру коммитов. Основные типы коммитов представлены в таблице ??.

Таблица 3.1: Типы коммитов Conventional Commits {#tbl:commit-types}

Тип	Описание
feat	Новая функциональность (соответствует MINOR в SemVer)
fix	Исправление ошибки (соответствует PATCH в SemVer)
docs	Изменения только в документации
style	Изменения форматирования (пробелы, отступы и т.д.)
refactor	Изменение кода без исправления ошибок и добавления функций
perf	Изменения, улучшающие производительность
test	Добавление или исправление тестов
build	Изменения системы сборки или внешних зависимостей

Тип	Описание
ci	Изменения конфигурации CI
chore	Прочие изменения, не затрагивающие исходный код

4 Выполнение лабораторной работы

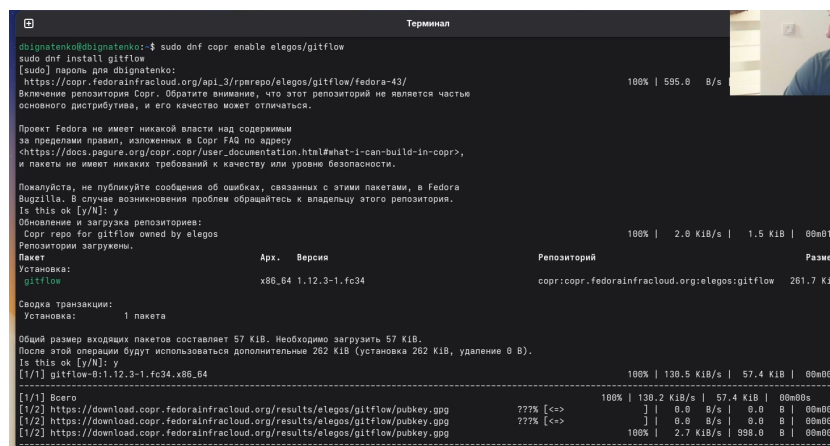
4.1 Установка программного обеспечения

4.1.1 Установка git-flow

Устанавливаю git-flow из коллекции репозитория Copr. Сначала подключаю репозиторий командой `dnf copr enable elegos/gitflow`, затем устанавливаю пакет командой `dnf install gitflow` (рис. 4.1).

```
sudo dnf copr enable elegos/gitflow
```

```
sudo dnf install gitflow
```



```
dbignatenko@dbignatenko:~$ sudo dnf copr enable elegos/gitflow
[sudo] пароль для dbignatenko:
https://copr.fedorainfracloud.org/api_3/rpms/elegos/gitflow/fedora-43/
Включение репозитория Copr. Обратите внимание, что этот репозиторий не является частью
основного дистрибутива, и его качество может отличаться.

Проект Fedora не имеет никакой власти над содержимым
за пределами правил, изложенных в Copr FAQ по адресу
<https://docs.pagure.org/copr.copr/user_documentation.html#what-i-can-build-in-copr>,
и пакеты не имеют никаких требований к качеству или уровню безопасности.

Пожалуйста, не публикуйте сообщения об ошибках, связанных с этими пакетами, в Fedora
Bugzilla. В случае возникновения проблем обращайтесь к владельцу этого репозитория.
Is this ok [y/N]: y
Обновление и загрузка репозитория:
Copr repo for gitflow owned by elegos
Репозиторий загружен.
Пакет      Арх.  Версия      Репозиторий      Размер
Установка:
gitflow    x86_64 1.12.3-1.fc34  copr:copr.fedorainfracloud.org:elegos:gitflow 261.7 KiB

Сводка транзакции:
Установка: 1 пакета

Общий размер входящих пакетов составляет 57 KiB. Необходимо загрузить 57 KiB.
После этой операции будут использоваться дополнительные 262 KiB (установка 262 KiB, удаление 0 B).
Is this ok [y/N]: y
[1/1] gitflow-0:1.12.3-1.fc34.x86_64 100% | 130.5 KiB/s | 57.4 KiB | 00m00s

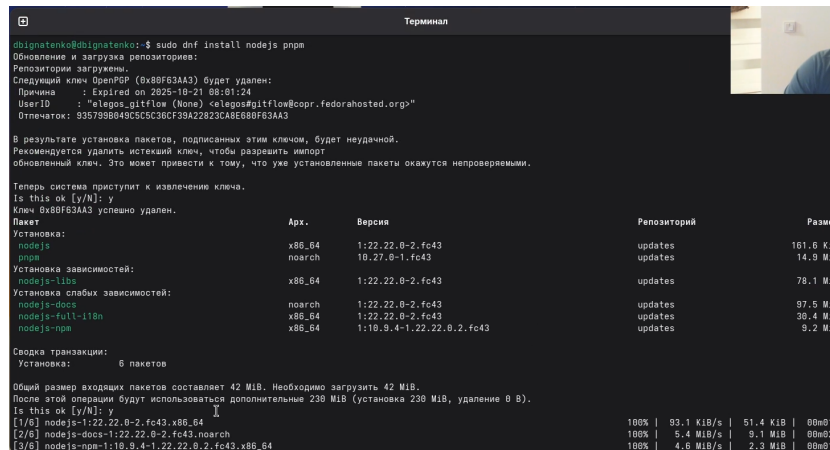
[1/1] Всего 100% | 130.2 KiB/s | 57.4 KiB | 00m00s
[1/2] https://download.copr.fedorainfracloud.org/results/elegos/gitflow/pubkey.gpg 777% [==>] | 0.0 B/s | 0.0 B | 00m00s
[1/2] https://download.copr.fedorainfracloud.org/results/elegos/gitflow/pubkey.gpg 777% [==>] | 0.0 B/s | 0.0 B | 00m00s
[1/2] https://download.copr.fedorainfracloud.org/results/elegos/gitflow/pubkey.gpg 100% | 2.7 KiB/s | 988.0 B | 00m00s
```

Рис. 4.1: Установка git-flow

4.1.2 Установка Node.js

Устанавливаю Node.js иригид `pnpm`, которые необходимы для работы с инструментами семантического версионирования и общепринятых коммитов (рис. 4.2).

```
sudo dnf install nodejs pnpm
```



```
dbignatenko@dbignatenko:~$ sudo dnf install nodejs pnpm
Обновление и загрузка репозитория:
Репозитории загружены.
Следующий ключ OpenPGP (0x8BF63AA3) будет удален:
Причина: Expired on 2025-10-21 08:01:24
User-ID: "elegos_github (None) <elegos@github.com>"
Отпечаток: 935798B048C5C36CF39A22823C48E680F63AA3

В результате установка пакетов, подписанных этим ключом, будет неудачной.
Рекомендуется удалить истекший ключ, чтобы разрешить импорт
обновленный ключ. Это может привести к тому, что уже установленные пакеты окажутся непроверяемыми.

Теперь система приступит к извлечению ключа.
Is this ok [y/N]: y
Ключ 0x8BF63AA3 успешно удален.

Пакет                                Арх.      Версия      Репозиторий      Размер
-----
Установка:
nodejs                               x86_64    1:22.22.0-2.fc43 updates          161.6 KiB
pnpm                                  noarch    10.27.0-1.fc43 updates           14.9 MiB
Установка зависимостей:
nodejs                               x86_64    1:22.22.0-2.fc43 updates           78.1 MiB
Установка слабых зависимостей:
nodejs-docs                          noarch    1:22.22.0-2.fc43 updates           97.5 MiB
nodejs-fault-118n                    x86_64    1:22.22.0-2.fc43 updates            30.4 MiB
nodejs-pnpm                           x86_64    1:10.9.4-1.22.22.0-2.fc43 updates            9.2 MiB

Сводка транзакции:
Установка:      6 пакетов

Общий размер входящих пакетов составляет 42 MiB. Необходимо загрузить 42 MiB.
После этой операции будут использоваться дополнительные 230 MiB (установка 230 MiB, удаление 0 B).
Is this ok [y/N]: y
[1/6] nodejs-1:22.22.0-2.fc43.x86_64 100% | 93.1 KiB/s | 51.4 KiB | 00m01s
[2/6] nodejs-docs-1:22.22.0-2.fc43.noarch 100% | 5.4 MiB/s | 9.1 MiB | 00m02s
[3/6] nodejs-pnpm-1:10.9.4-1.22.22.0-2.fc43.x86_64 100% | 4.6 MiB/s | 2.3 MiB | 00m01s
```

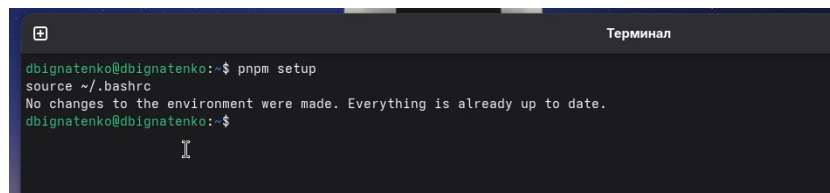
Рис. 4.2: Установка Node.js и `pnpm`

4.1.3 Настройка Node.js

Выполняю настройку `pnpm` для добавления каталога с исполняемыми файлами в переменную `PATH`. Команда `pnpm setup` уже была выполнена ранее (рис. 4.3).

```
pnpm setup
```

```
source ~/.bashrc
```



```
dbignatenko@dbignatenko:~$ pnpm setup
source ~/.bashrc
No changes to the environment were made. Everything is already up to date.
dbignatenko@dbignatenko:~$
```

Рис. 4.3: Настройка `pnpm`

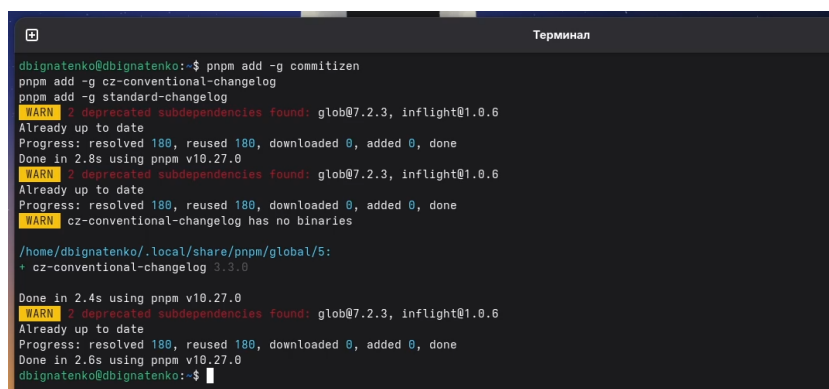
4.1.4 Установка commitizen и standard-changelog

Устанавливаю глобально пакеты commitizen, cz-conventional-changelog и standard-changelog для работы с общепринятыми коммитами и генерации журнала изменений (рис. 4.4).

```
pnpm add -g commitizen
```

```
pnpm add -g cz-conventional-changelog
```

```
pnpm add -g standard-changelog
```



```
dbignatenko@dbignatenko:~$ pnpm add -g commitizen
pnpm add -g cz-conventional-changelog
pnpm add -g standard-changelog
WARN 2 deprecated subdependencies found: glob@7.2.3, inflight@1.0.6
Already up to date
Progress: resolved 180, reused 180, downloaded 0, added 0, done
Done in 2.8s using pnpm v10.27.0
WARN 2 deprecated subdependencies found: glob@7.2.3, inflight@1.0.6
Already up to date
Progress: resolved 180, reused 180, downloaded 0, added 0, done
WARN cz-conventional-changelog has no binaries

/home/dbignatenko/.local/share/pnpm/global/5:
+ cz-conventional-changelog 3.3.0

Done in 2.4s using pnpm v10.27.0
WARN 2 deprecated subdependencies found: glob@7.2.3, inflight@1.0.6
Already up to date
Progress: resolved 180, reused 180, downloaded 0, added 0, done
Done in 2.6s using pnpm v10.27.0
dbignatenko@dbignatenko:~$
```

Рис. 4.4: Установка commitizen и standard-changelog

4.2 Создание репозитория на GitHub

Создаю новый публичный репозиторий на GitHub с названием git-extended для практической работы с git-flow и conventional commits (рис. 4.5).

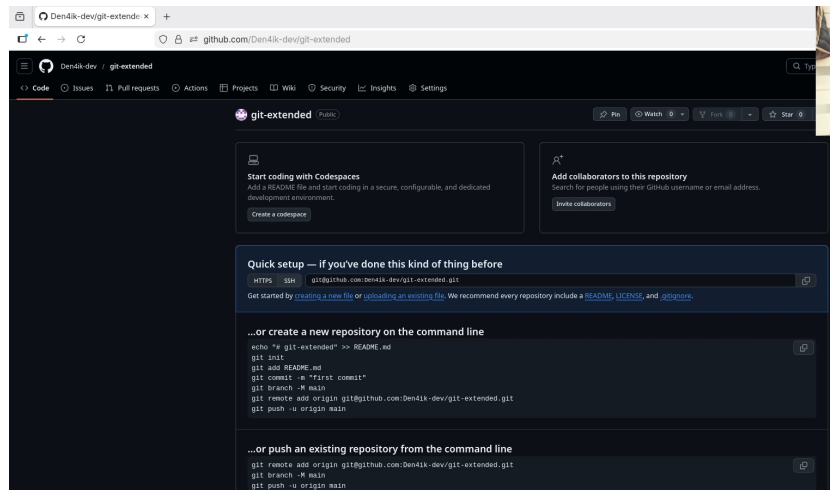


Рис. 4.5: Страница созданного репозитория на GitHub

4.3 Локальная инициализация репозитория

Создаю локальный репозиторий, делаю первый коммит и отправляю его на GitHub (рис. 4.6).

```
mkdir ~/git-extended
cd ~/git-extended
git init
git commit --allow-empty -m "first commit"
git remote add origin git@github.com:username/git-extended.git
git push -u origin master
```

```
dbignatenko@dbignatenko:~$ mkdir ~/git-extended
cd ~/git-extended
git init
git commit --allow-empty -m "first commit"
git remote add origin git@github.com:Den4ik-dev/git-extended.git
git push -u origin master
mkdir: невозможно создать каталог «/home/dbignatenko/git-extended»: Файл существует
Переинициализирован существующий репозиторий Git в /home/dbignatenko/git-extended/.git/
[master c2e3e1d] first commit
1 file changed, 18 insertions(+)
create mode 100644 package.json
error: внешний репозиторий origin уже существует
Перечисление объектов: 5, готово.
Подсчет объектов: 100% (5/5), готово.
При сжатии изменений используется до 6 потоков
Сжатие объектов: 100% (3/3), готово.
Запись объектов: 100% (5/5), 1.95 KiB | 1.95 MiB/s, готово.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Den4ik-dev/git-extended.git
 * [new branch] master -> master
branch 'master' set up to track 'origin/master'.
dbignatenko@dbignatenko:~/git-extended$
```

Рис. 4.6: Локальная инициализация и отправка на GitHub

4.4 Настройка conventional commits

4.4.1 Инициализация pnpm

Инициализирую конфигурацию Node.js в репозитории командой `pnpm init`. Указываю название пакета `git-extended` и лицензию `CC-BY-4.0` (рис. 4.7).

```
pnpm init
```

```
dbignatenko@dbignatenko:~/git-extended$ pnpm init
Wrote to /home/dbignatenko/git-extended/package.json

{
  "name": "git-extended",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "packageManager": "pnpm@10.27.0"
}
dbignatenko@dbignatenko:~/git-extended$
```

Рис. 4.7: Инициализация pnpm

4.4.2 Редактирование package.json

Открываю файл package.json и добавляю блок config для настройки commitizen с использованием cz-conventional-changelog (рис. 4.8).

```
{
  "name": "git-extended",
  "version": "1.0.0",
  "description": "Git repo for educational purposes",
  "main": "index.js",
  "repository": "git@github.com:username/git-extended.git",
  "author": "Имя Фамилия <email@gmail.com>",
  "license": "CC-BY-4.0",
  "config": {
    "commitizen": {
      "path": "cz-conventional-changelog"
    }
  }
}
```

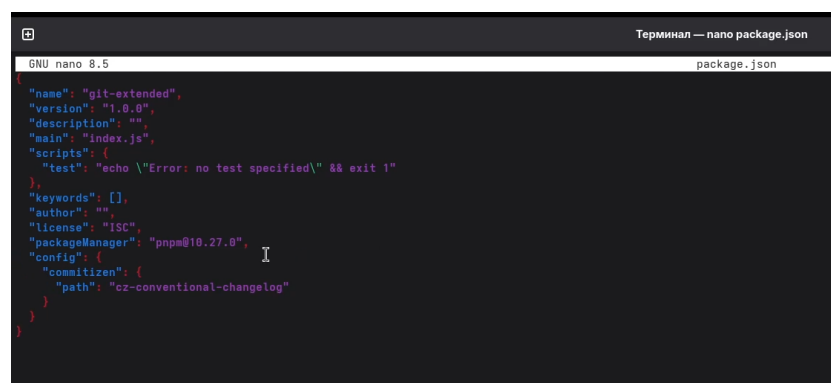


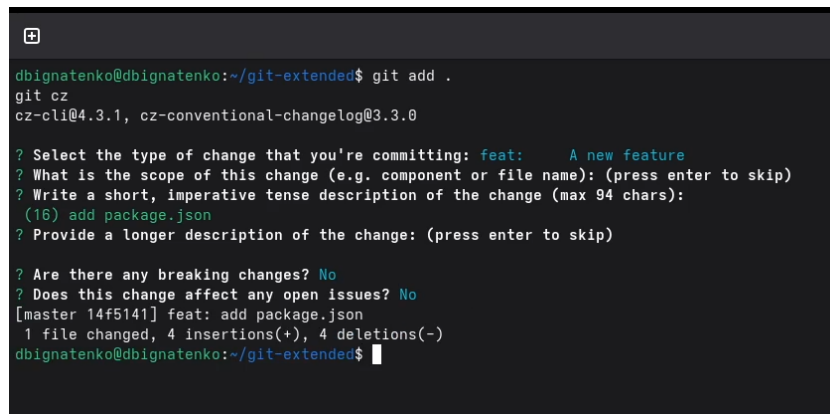
Рис. 4.8: Содержимое package.json

4.4.3 Первый коммит через git cz

Добавляю файлы в индекс и выполняю коммит с помощью интерактивного интерфейса git cz. Выбираю тип коммита feat и указываю описание add package.json (рис. 4.9).

```
git add .
```

```
git cz
```



```
dbignatenko@dbignatenko:~/git-extended$ git add .
git cz
cz-cli@4.3.1, cz-conventional-changelog@3.3.0

? Select the type of change that you're committing: feat:      A new feature
? What is the scope of this change (e.g. component or file name): (press enter to skip)
? Write a short, imperative tense description of the change (max 94 chars):
  (16) add package.json
? Provide a longer description of the change: (press enter to skip)

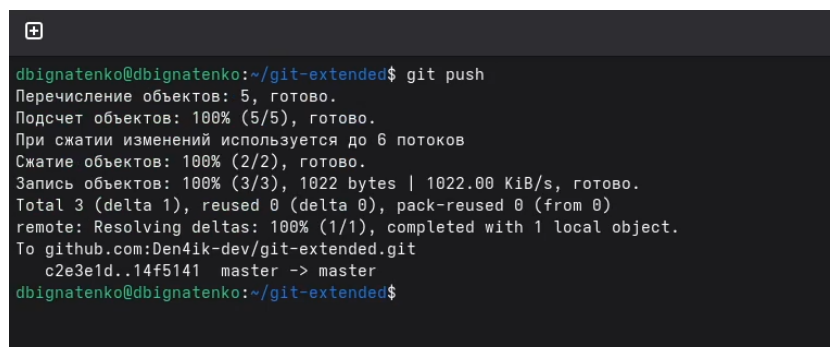
? Are there any breaking changes? No
? Does this change affect any open issues? No
[master 14f5141] feat: add package.json
 1 file changed, 4 insertions(+), 4 deletions(-)
dbignatenko@dbignatenko:~/git-extended$
```

Рис. 4.9: Интерфейс git cz

4.4.4 Отправка на GitHub

Отправляю изменения на удалённый репозиторий (рис. 4.10).

```
git push
```



```
dbignatenko@dbignatenko:~/git-extended$ git push
Перечисление объектов: 5, готово.
Подсчет объектов: 100% (5/5), готово.
При сжатии изменений используется до 6 потоков
Сжатие объектов: 100% (2/2), готово.
Запись объектов: 100% (3/3), 1022 bytes | 1022.00 KiB/s, готово.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:Den4ik-dev/git-extended.git
 c2e3e1d..14f5141  master -> master
dbignatenko@dbignatenko:~/git-extended$
```

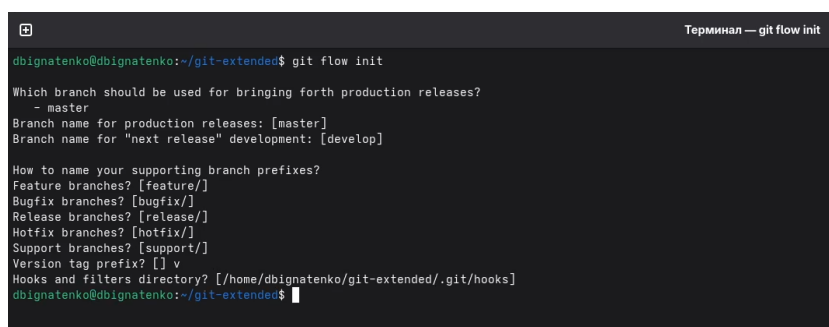
Рис. 4.10: Отправка изменений на GitHub

4.5 Инициализация git-flow

4.5.1 Инициализация структуры git-flow

Инициализирую git-flow в репозитории. На все вопросы принимаю значения по умолчанию, кроме префикса для тегов версий — указываю v (рис. 4.11).

```
git flow init
```



```
dbignatenko@dbignatenko:~/git-extended$ git flow init
Which branch should be used for bringing forth production releases?
- master
Branch name for production releases: [master]
Branch name for "next release" development: [develop]

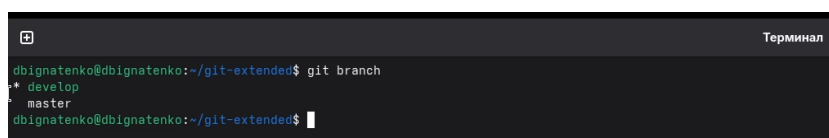
How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? [] v
Hooks and filters directory? [/home/dbignatenko/git-extended/.git/hooks]
dbignatenko@dbignatenko:~/git-extended$
```

Рис. 4.11: Инициализация git-flow

4.5.2 Проверка текущей ветки

Проверяю, что нахожусь на ветке develop после инициализации git-flow (рис. 4.12).

```
git branch
```



```
dbignatenko@dbignatenko:~/git-extended$ git branch
* develop
  master
dbignatenko@dbignatenko:~/git-extended$
```

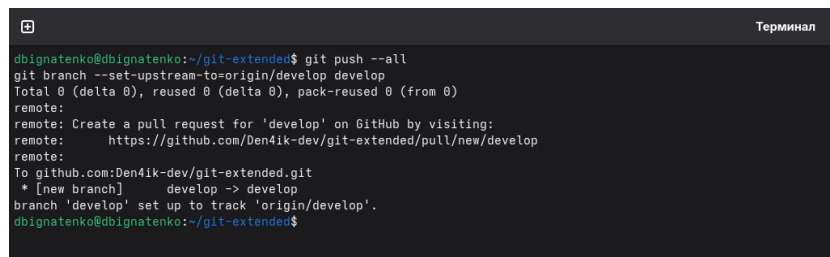
Рис. 4.12: Проверка текущей ветки

4.5.3 Отправка всех веток на GitHub

Загружаю все ветки в удалённый репозиторий и устанавливаю внешнюю ветку как вышестоящую для ветки develop (рис. 4.13).

```
git push --all
```

```
git branch --set-upstream-to=origin/develop develop
```



```
dbignatenko@dbignatenko:~/git-extended$ git push --all
git branch --set-upstream-to=origin/develop develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/Den4ik-dev/git-extended/pull/new/develop
remote:
To github.com:Den4ik-dev/git-extended.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.
dbignatenko@dbignatenko:~/git-extended$
```

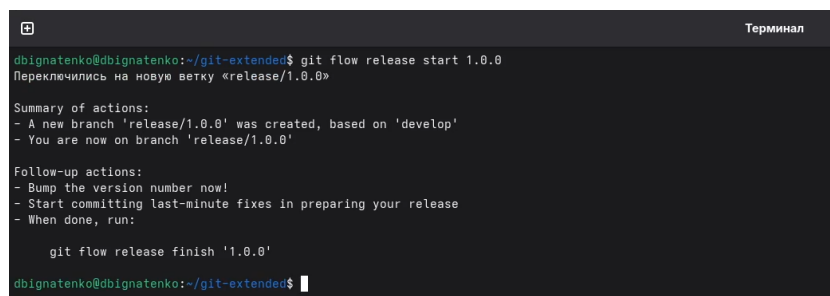
Рис. 4.13: Отправка всех веток на GitHub

4.6 Создание релиза 1.0.0

4.6.1 Создание ветки релиза

Создаю ветку релиза для версии 1.0.0 (рис. 4.14).

```
git flow release start 1.0.0
```



```
dbignatenko@dbignatenko:~/git-extended$ git flow release start 1.0.0
Переключились на новую ветку «release/1.0.0»

Summary of actions:
- A new branch 'release/1.0.0' was created, based on 'develop'
- You are now on branch 'release/1.0.0'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

    git flow release finish '1.0.0'

dbignatenko@dbignatenko:~/git-extended$
```

Рис. 4.14: Создание ветки релиза 1.0.0

4.6.2 Создание журнала изменений

Создаю файл CHANGELOG.md с помощью утилиты standard-changelog с флагом --first-release для первого релиза (рис. 4.15).

```
standard-changelog --first-release
git add CHANGELOG.md
git commit -am 'chore(site): add changelog'
```

```
dbignatenko@dbignatenko:~/git-extended$ standard-changelog --first-release
dbignatenko@dbignatenko:~/git-extended$ git add CHANGELOG.md
git commit -am 'chore(site): add changelog'
[release/1.0.0 dbf3ab2] chore(site): add changelog
1 file changed, 6 insertions(+)
create mode 100644 CHANGELOG.md
dbignatenko@dbignatenko:~/git-extended$ cat CHANGELOG.md
# 1.0.0 (2020-05-05)

## Features

* add package.json ([14f5141](https://github.com/Den4ik-dev/git-extended/commit/14f5141be8992b84ef2e9540fc68a5c64472580c))
dbignatenko@dbignatenko:~/git-extended$
```

Рис. 4.15: Содержимое CHANGELOG.md

4.6.3 Завершение релиза

Завершаю работу над релизом, сливая ветку release в master и develop (рис. 4.16).

```
git flow release finish 1.0.0
```

```
dbignatenko@dbignatenko:~/git-extended$ git flow release finish 1.0.0
Переключились на ветку «master»
Эта ветка соответствует «origin/master».
Merge made by the 'ort' strategy.
CHANGELOG.md | 6 +++++
1 file changed, 6 insertions(+)
create mode 100644 CHANGELOG.md
Уже на «master»
Ваша ветка опережает «origin/master» на 2 коммита.
(используйте «git push», чтобы опубликовать ваши локальные коммиты)
Переключились на ветку «develop»
Эта ветка соответствует «origin/develop».
Merge made by the 'ort' strategy.
CHANGELOG.md | 6 +++++
1 file changed, 6 insertions(+)
create mode 100644 CHANGELOG.md
Ветка release/1.0.0 удалена (была dbf3ab2).

Summary of actions:
- Release branch 'release/1.0.0' has been merged into 'master'
- The release was tagged 'v1.0.0'
- Release tag 'v1.0.0' has been back-merged into 'develop'
- Release branch 'release/1.0.0' has been locally deleted
- You are now on branch 'develop'

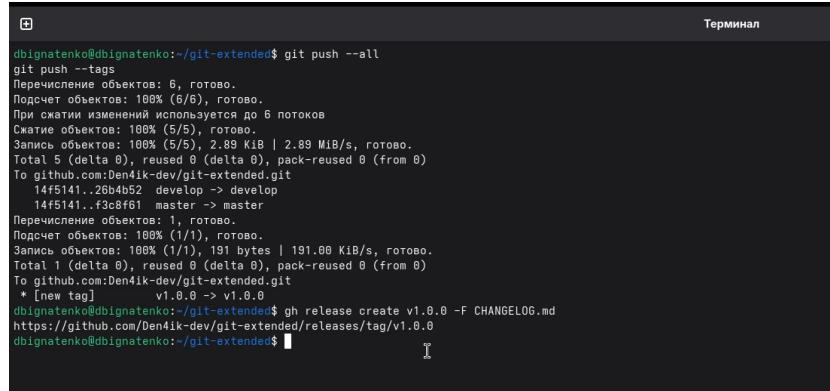
dbignatenko@dbignatenko:~/git-extended$
```

Рис. 4.16: Завершение релиза 1.0.0

4.6.4 Отправка релиза на GitHub

Отправляю все ветки и теги на GitHub, а также создаю релиз с помощью утилиты gh (рис. 4.17).

```
git push --all
git push --tags
gh release create v1.0.0 -F CHANGELOG.md
```



```
dbignatenko@dbignatenko:~/git-extended$ git push --all
git push --tags
Перечисление объектов: 6, готово.
Подсчет объектов: 100% (6/6), готово.
При сжатии изменений используется до 6 потоков
Сжатие объектов: 100% (5/5), готово.
Запись объектов: 100% (5/5), 2.89 KiB | 2.89 MiB/s, готово.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Den4ik-dev/git-extended.git
 14f5141..26b4b52 develop -> develop
 14f5141..f3c8f61 master -> master
Перечисление объектов: 1, готово.
Подсчет объектов: 100% (1/1), готово.
Запись объектов: 100% (1/1), 191 bytes | 191.00 KiB/s, готово.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Den4ik-dev/git-extended.git
 * [new tag]         v1.0.0 -> v1.0.0
dbignatenko@dbignatenko:~/git-extended$ gh release create v1.0.0 -F CHANGELOG.md
https://github.com/Den4ik-dev/git-extended/releases/tag/v1.0.0
dbignatenko@dbignatenko:~/git-extended$
```

Рис. 4.17: Отправка релиза на GitHub

4.6.5 Страница релизов на GitHub

Проверяю страницу релизов на GitHub — релиз v1.0.0 успешно создан (рис. 4.18).

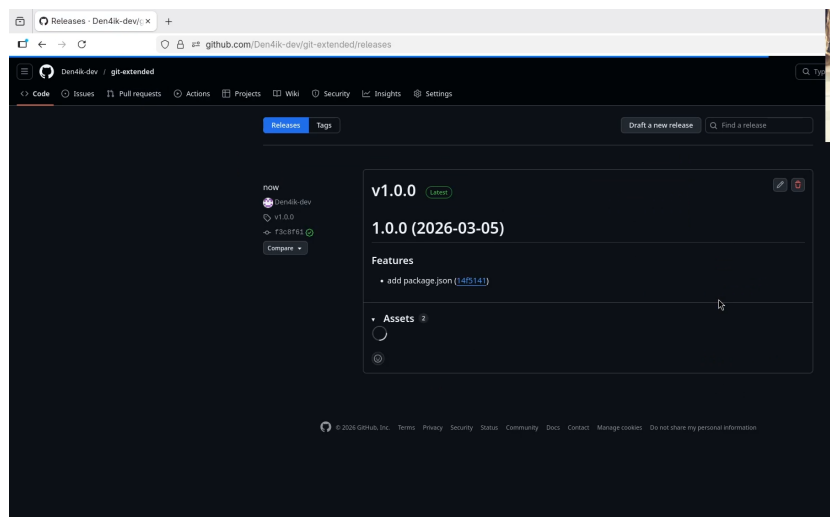


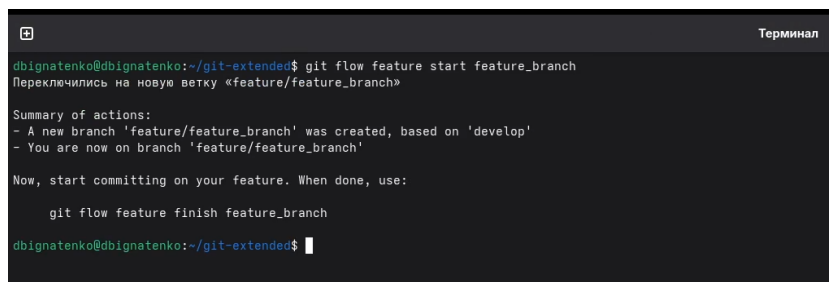
Рис. 4.18: Страница релизов на GitHub

4.7 Работа с feature-веткой

4.7.1 Создание feature-ветки

Создаю новую функциональную ветку для разработки нового функционала (рис. 4.19).

```
git flow feature start feature_branch
```



```
dbignatenko@dbignatenko:~/git-extended$ git flow feature start feature_branch
Переключились на новую ветку «feature/feature_branch»

Summary of actions:
- A new branch 'feature/feature_branch' was created, based on 'develop'
- You are now on branch 'feature/feature_branch'

Now, start committing on your feature. When done, use:

    git flow feature finish feature_branch

dbignatenko@dbignatenko:~/git-extended$
```

Рис. 4.19: Создание feature-ветки

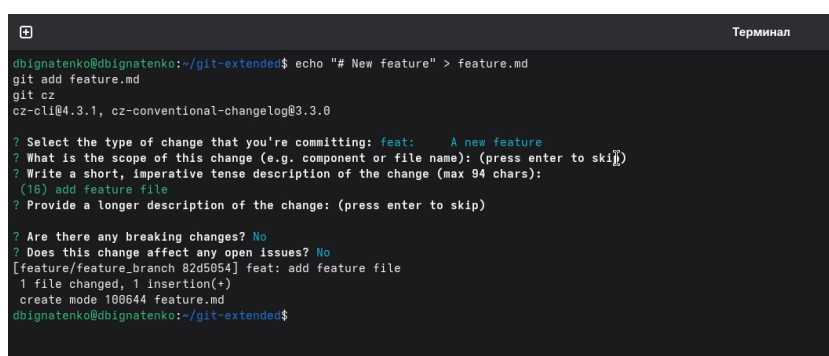
4.7.2 Добавление нового файла

Создаю новый файл feature.md и выполняю коммит через git cz с типом feat (рис. 4.20).

```
echo "# New feature" > feature.md
```

```
git add feature.md
```

```
git cz
```



```
dbignatenko@dbignatenko:~/git-extended$ echo "# New feature" > feature.md
git add feature.md
git cz
cz-cli@4.3.1, cz-conventional-changelog@3.3.0

? Select the type of change that you're committing: feat: A new feature
? What is the scope of this change (e.g. component or file name): (press enter to skip)
? Write a short, imperative tense description of the change (max 94 chars):
  (16) add feature file
? Provide a longer description of the change: (press enter to skip)

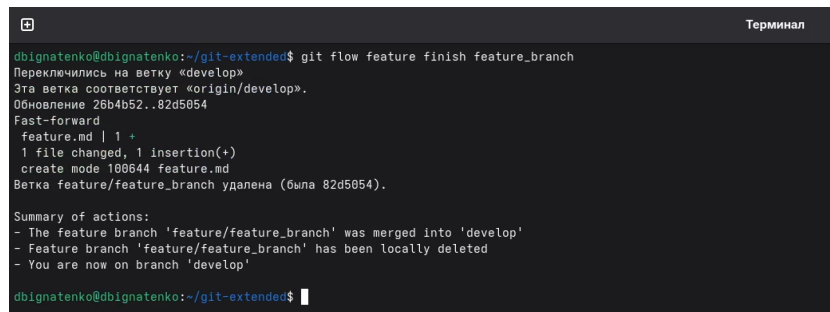
? Are there any breaking changes? No
? Does this change affect any open issues? No
[feature/feature_branch 82d5864] feat: add feature file
1 file changed, 1 insertion(+)
create mode 100644 feature.md
dbignatenko@dbignatenko:~/git-extended$
```

Рис. 4.20: Коммит через git cz

4.7.3 Завершение feature-ветки

Завершаю работу над feature-веткой, сливая её с веткой develop (рис. 4.21).

```
git flow feature finish feature_branch
```



```
dbignatenko@dbignatenko:~/git-extended$ git flow feature finish feature_branch
Переключились на ветку «develop»
Эта ветка соответствует «origin/develop».
Обновление 26b4b52..82d5054
Fast-forward
 feature.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 feature.md
Ветка feature/feature_branch удалена (была 82d5054).

Summary of actions:
- The feature branch 'feature/feature_branch' was merged into 'develop'
- Feature branch 'feature/feature_branch' has been locally deleted
- You are now on branch 'develop'

dbignatenko@dbignatenko:~/git-extended$
```

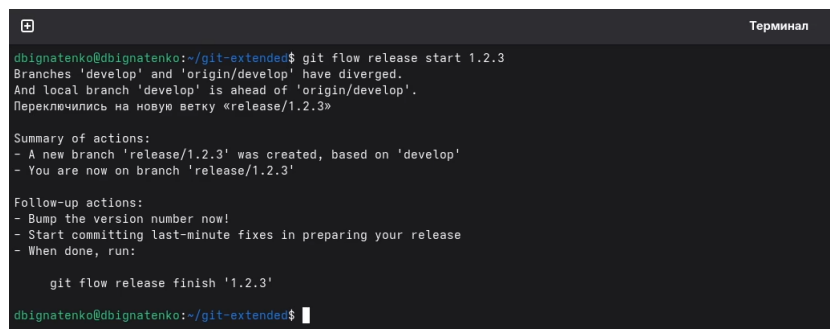
Рис. 4.21: Завершение feature-ветки

4.8 Создание релиза 1.2.3

4.8.1 Создание ветки релиза

Создаю ветку для нового релиза версии 1.2.3 (рис. 4.22).

```
git flow release start 1.2.3
```



```
dbignatenko@dbignatenko:~/git-extended$ git flow release start 1.2.3
Branches 'develop' and 'origin/develop' have diverged.
And local branch 'develop' is ahead of 'origin/develop'.
Переключились на новую ветку «release/1.2.3»

Summary of actions:
- A new branch 'release/1.2.3' was created, based on 'develop'
- You are now on branch 'release/1.2.3'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

    git flow release finish '1.2.3'

dbignatenko@dbignatenko:~/git-extended$
```

Рис. 4.22: Создание ветки релиза 1.2.3

4.8.2 Обновление версии в package.json

Обновляю номер версии в файле package.json, изменяя значение с 1.0.0 на 1.2.3 (рис. 4.23).



Рис. 4.23: Обновлённый package.json

4.8.3 Обновление журнала изменений

Обновляю файл CHANGELOG.md для нового релиза (рис. 4.24).

standard-changelog

git add CHANGELOG.md

git commit -am 'chore(site): update changelog'

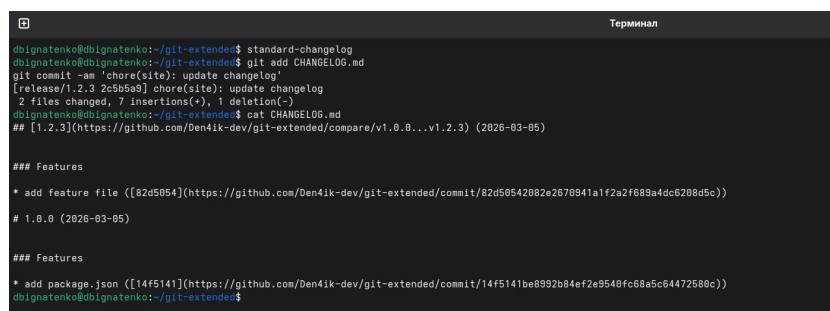


Рис. 4.24: Обновлённый CHANGELOG.md

4.8.4 Завершение релиза 1.2.3

Завершаю работу над релизом 1.2.3 (рис. 4.25).

git flow release finish 1.2.3

```
dbignatenko@dbignatenko:~/git-extended$ git flow release finish 1.2.3
Branches 'develop' and 'origin/develop' have diverged.
And local branch 'develop' is ahead of 'origin/develop'.
Переключились на ветку «master»
Эта ветка соответствует «origin/master».
Merge made by the 'ort' strategy.
CHANGELOG.md | 7 ++++++
feature.md    | 1 +
package.json | 2 +-
3 files changed, 9 insertions(+), 1 deletion(-)
create mode 100644 feature.md
Уже на «master»
Ваша ветка опережает «origin/master» на 5 коммитов.
(используйте «git push», чтобы опубликовать ваши локальные коммиты)
Переключились на ветку «develop»
Ваша ветка опережает «origin/develop» на 1 коммит.
(используйте «git push», чтобы опубликовать ваши локальные коммиты)
Merge made by the 'ort' strategy.
CHANGELOG.md | 7 ++++++
package.json | 2 +-
2 files changed, 8 insertions(+), 1 deletion(-)
Ветка release/1.2.3 удалена (была 2c5b5a9).

Summary of actions:
- Release branch 'release/1.2.3' has been merged into 'master'
- The release was tagged 'v1.2.3'
- Release tag 'v1.2.3' has been back-merged into 'develop'
- Release branch 'release/1.2.3' has been locally deleted
- You are now on branch 'develop'

dbignatenko@dbignatenko:~/git-extended$
```

Рис. 4.25: Завершение релиза 1.2.3

4.8.5 Отправка релиза на GitHub

Отправляю все изменения и создаю новый релиз на GitHub (рис. 4.26).

```
git push --all
```

```
git push --tags
```

```
gh release create v1.2.3 -F CHANGELOG.md
```

```
dbignatenko@dbignatenko:~/git-extended$ git push --all
git push --tags
gh release create v1.2.3 -F CHANGELOG.md
Перечисление объектов: 15, готово.
Подсчет объектов: 100% (15/15), готово.
При сжатии изменений используется до 6 потоков
Сжатие объектов: 100% (11/11), готово.
Запись объектов: 100% (12/12), 4.99 KiB | 2.50 MiB/s, готово.
Total 12 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To github.com:Den4ik-dev/git-extended.git
 26b4b52..d6c9f01 develop -> develop
 f3c8f61..5aca920 master -> master
Перечисление объектов: 1, готово.
Подсчет объектов: 100% (1/1), готово.
Запись объектов: 100% (1/1), 191 bytes | 191.00 KiB/s, готово.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Den4ik-dev/git-extended.git
 * [new tag]          v1.2.3 -> v1.2.3
https://github.com/Den4ik-dev/git-extended/releases/tag/v1.2.3
dbignatenko@dbignatenko:~/git-extended$
```

Рис. 4.26: Отправка релиза 1.2.3 на GitHub

4.8.6 Страница релизов с обоими релизами

Проверяю страницу релизов на GitHub — теперь доступны оба релиза: v1.0.0 и v1.2.3 (рис. 4.27).

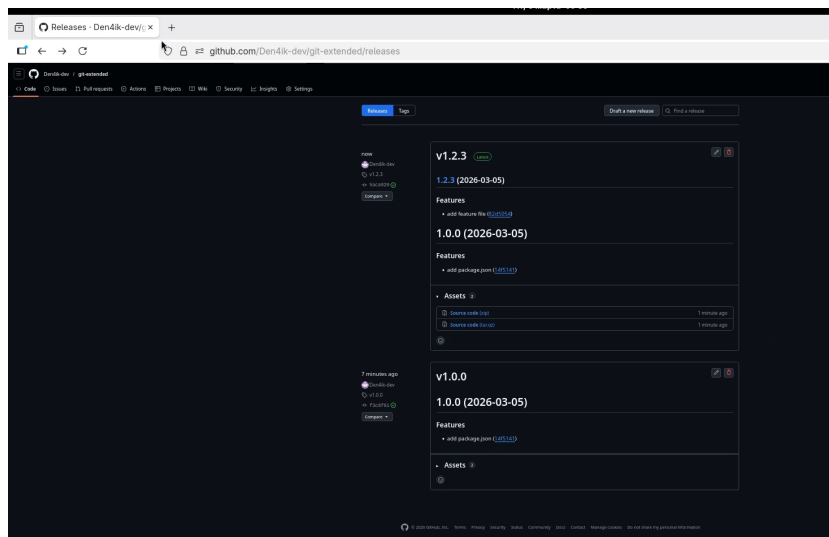


Рис. 4.27: Страница релизов с обоими релизами

5 Выводы

В ходе выполнения лабораторной работы получил навыки правильной работы с репозиториями git. Освоил методологию Gitflow Workflow для организации процесса разработки с использованием веток master, develop, feature и release. Изучил принципы семантического версионирования и научился применять спецификацию Conventional Commits для стандартизации сообщений коммитов. Настроил инструменты commitizen и standard-changelog для автоматизации процесса создания коммитов и генерации журнала изменений.

Список литературы